LangGraph StateGraph 학습 노트

날짜: 2026-05-22
복습 예정: D+1, D+3, D+7

한 줄 요약

에이전트가 어떤 순서로 일할지 "흐름도"를 코드로 그리는 도구.

---

1. 비유

공장 컨베이어 벨트에 비유:

```
[부품 투입] -> [조립] -> [검수] -> [포장] -> [출하]
```

각 단계(노드)가 끝나야 다음 단계로 넘어가고,
현재 상태(부품이 어디까지 왔는지)를 State가 기억한다.

Dev 2에서:

```
[run_tools] -> [llm_judge] -> [vector_search] -> [store]
  CLI 실행       LLM 판단       유사 이슈 검색     DB 저장
```

---

2. 개념

> LangGraph는 에이전트의 상태(State)와 흐름(Graph)을 정의하는 프레임워크.
> 노드(Node) = 작업 단위, 엣지(Edge) = 순서, 상태(State) = 노드 간 공유 데이터.

핵심 3요소:

요소	**역할**	**Dev 2 예시**
State	노드 간 공유되는 데이터	[Dev2State] (static_raw, enriched 등)
Node	실제 작업을 수행하는 함수	[run_tools_node], [llm_judge_node]
Edge	노드 간 실행 순서	[run_tools -> llm_judge -> vector_search -> store]

---

3. 왜 쓰나

- 없으면: 함수를 직접 순서대로 호출 -> 상태 관리가 복잡해짐, 중간에 멈추거나 재시작 불가
- 있으면: 상태를 자동으로 관리, 노드별로 독립 테스트 가능, LangSmith로 각 노드 추적 가능

---

4. 코드 (Dev 2)

```
[python]
# graph.py — LangGraph StateGraph 조립
from langgraph.graph import StateGraph, END
from typing import TypedDict

# State 정의: 노드 간 공유되는 데이터 구조
class Dev2State(TypedDict):
    payload: dict          # 입력 (PR diff, 개발자 정보)
    static_raw: list       # Pylint/radon 원본 결과
    security_raw: list    # Bandit/pip-audit 원본 결과
    enriched: list         # LLM 판단 후 결과
    final_output: list    # Dev4로 전달할 최종 결과

def build_graph():
    g = StateGraph(Dev2State)

    # 노드 등록
    g.add_node("run_tools", run_tools_node)
    g.add_node("llm_judge", llm_judge_node)
    g.add_node("vector_search", vector_search_node)
    g.add_node("store", store_node)

    # 엣지 (순서) 정의
    g.set_entry_point("run_tools")            # 시작점
    g.add_edge("run_tools", "llm_judge")
    g.add_edge("llm_judge", "vector_search")
    g.add_edge("vector_search", "store")
    g.add_edge("store", END)                  # 종료

    return g.compile()

# 실행
graph = build_graph()
result = await graph.ainvoke(initial_state)
```

흐름 시각화:

```
Dev2State 공유 ->->->->->->->->->->->->->->->->->->->->->->->->->->->
    v
[run_tools] -> static_raw, security_raw 채움
    v
[llm_judge] -> enriched 채움 (LLM severity 판단)
    v
[vector_search] -> enriched에 similar_issues 추가
    v
[store] -> final_output 채움 + DB 저장
    v
END
```

복습 퀴즈

1. Q: LangGraph의 State란 무엇인가?

A: (노드 간 공유되는 데이터 구조. TypedDict로 정의)

2. Q: Dev 2의 LangGraph 노드 4개는?
   A: (run_tools -> llm_judge -> vector_search -> store)
3. Q: 일반 함수 호출 대신 LangGraph를 쓰는 이유는?
   A: (상태 관리 자동화, 노드별 독립 테스트, LangSmith 추적 가능)